

Image Classification on Arduino Board

Project Notes

OV7675 Camera Test

The availability of OV7675 camera on Arduino board can be tested by getting the hexadecimal raw data through debugging messages and putting the data into the OV7675 image viewer in [2].

We replaced “HEXADECIMAL_BYTES” data with our captured image data. For the next image viewing part, the code is shown below (slightly modified from the original one):

```
# Reformat the bytes into an image
raw_bytes = np.array(HEXADECIMAL_BYTES, dtype="u2") # originally "i2" for the second variable;
fixed
image = np.zeros((len(raw_bytes),3), dtype=int) # to prevent image size mismatch, use
(144*176,3) for the first variable

# Loop through all of the pixels and form the image
for i in range(len(raw_bytes)):
    #Read 16-bit pixel
    pixel = struct.unpack('>H', raw_bytes[i])[0] # originally ">h" for the first variable; fixed

    #Convert RGB565 to RGB 24-bit
    r = ((pixel >> 11) & 0x1f) << 3;
    g = ((pixel >> 5) & 0x3f) << 2;
    b = ((pixel >> 0) & 0x1f) << 3;
    image[i] = [r,g,b]

image = np.reshape(image,(48, 48,3)) #QCIF resolution; set according to the real image size

# Show the image
plt.imshow(image)
plt.show()
```

An image will be plotted below this box. Examples of images are shown below.



Connecting Arduino Board and OV7675 Camera to Edge Impulse

We connect the Arduino board with OV7675 camera to Edge Impulse platform to obtain the image dataset for training the classification model. It is done with the following steps (on Windows 11):

- (1). Install Node.js;
- (2). In cmd.exe, enter the command “npm install -g edge-impulse-cli”;
- (3). Upload the Edge Impulse firmware to the Arduino board. The package for the firmware is given in [1];
- (4). In cmd.exe, enter “edge-impulse-daemon” for Arduino board detection by Edge Impulse cloud server. Note that Arduino IDE should be closed to make sure only Edge Impulse is using the port. Logging in and selecting workspace are required for the first time.
- (5). If we want to switch the project the Arduino board and OV7675 camera work for, we should enter “edge-impulse-daemon –clean” for the re-connection.

Data Acquisition and Model Training

We trained an image classification model that contains three labels for images: A, B and C. The dataset contains 120 photos, 40 for each label. Examples of images for each label are shown below.



The captured images are of 120*96 pixels and RGB colors, but then when we create impulse, we squash the images into 48*48 pixels, and thus the number of features for each image is $48*48*3=6912$. We added “Image” as the processing block, and “Classification” as the learning block. Note that we should avoid tagging, training or recognizing this project as an “object detection” project in order to avoid labeling errors (we cannot label the images directly; instead, we have to select a bounding box on the image and label that box). After training, we get a model with 95.8% accuracy and 0.28 loss.

Live Classification

After training the image classification model on Edge Impulse, we can immediately use it through “Impulse design -> Live Classification”, capture real-time image with OV7675 camera, and classify the image with this model, as shown in the picture below. Each image captured here will be a new entry for “Test” image data in “Data acquisition”, and the inaccurate label can be changed for later reference in model testing.

Model Deployment on Arduino IDE

The source library for this model is ready via “Impulse Design -> Deployment”. The deployment option is selected as “Arduino library”, and we can download the library and integrate it into the Edge Impulse firmware for Arduino IDE [1].

在 src 中搜索						
名称	类型	压缩大小	密码保护	大小	比率	修改日期
edge-impulse-sdk	文件夹					2025/6/8 0:47
firmware-sdk	文件夹					2025/6/8 0:47
ingestion-sdk-c	文件夹					2025/6/8 0:47
ingestion-sdk-platform	文件夹					2025/6/8 0:47
mbdttls_hmac_sha256_sw	文件夹					2025/6/8 0:47
model-parameters	文件夹					2025/6/8 0:47
sensors	文件夹					2025/6/8 0:47
tfllite-model	文件夹					2025/6/8 0:47
setup.cpp	C++ 源文件	1 KB	否	3 KB	59%	2025/6/8 0:47
setup.h	H 文件	2 KB	否	3 KB	51%	2025/6/8 0:47

The source library for building the Arduino project is downloaded via “Impulse Design -> Deployment” on Edge Impulse. The deployment option is selected as “Arduino library”. Upon downloading, its subdirectories are shown below.

在 src 中搜索						
名称	类型	压缩大小	密码保护	大小	比率	修改日期
edge-impulse-sdk	文件夹					2025/6/12 5:07
model-parameters	文件夹					2025/6/12 5:07
tfllite-model	文件夹					2025/6/12 5:07
Xiaohe-project-1_inferencing.h	H 文件	1 KB	否	2 KB	52%	2025/6/12 5:07

We replaced the three subfolders in firmware library folder with those of the same name in project source library folder, as well as copying the header file (named “Xiaohe-project-1_inferencing.h” in our project) to the firmware library folder. Then, we packaged the firmware library src folder to a zip file and imported it into Arduino library. It is then found in Arduino library path as shown below. Subsequent modifications are done directly for files in this path.

在 src 中搜索						
名称	修改日期	类型	大小			
edge-impulse-sdk	2025/6/13 18:36	文件夹				
examples	2025/6/13 18:36	文件夹				
firmware-sdk	2025/6/13 18:36	文件夹				
ingestion-sdk-c	2025/6/13 18:36	文件夹				
ingestion-sdk-platform	2025/6/13 18:36	文件夹				
mbdttls_hmac_sha256_sw	2025/6/13 18:36	文件夹				
model-parameters	2025/6/13 18:36	文件夹				
sensors	2025/6/13 18:36	文件夹				
tfllite-model	2025/6/13 18:36	文件夹				
Xiaohe-project-1_inferencing.h	2025/6/13 18:36	H 文件	2 KB			

Problems and Solutions

- Path errors: “files not found” reported, caused by failure of finding files intended to include.
 - Solution: a “boards.local.txt” file is given in firmware library containing include path information. The file is copied to the directory “C:\Users\ (username)\AppData\Local\Arduino15\packages\arduino\hardware\mbed_nano\4.3.1\”, and in the file “platform.txt” under the same directory, those paths are added to after “compiler.cpp.extra_flags=” Note that the file and path copying needs to be done

every time the Arduino IDE is upgraded. For further errors caused by include paths, we just manually edited include paths to absolute paths regarding src folder.

- Multiple definition errors: they are mainly caused by providing implementations of functions in .h files.
 - Solution: add “inline” at the declaration of the function. For some functions that contain “extern “C”” at the declaration, we move the implementation to the respective .cpp file (create one if the file does not exist), only keeping the prototype in the .h file.
- Porting related errors: there are multiple types of errors (multiple definition, undefined reference, redefinition, etc.) caused by functions defined in “ei_classifier_porting.h”.
 - Solution: there are multiple versions of “ei_classifier_porting.cpp” for different platforms (android, ambiq, arduino, etc.). In ei_classifier_porting.h, we set the flag “EI_PORTING_ARDUINO” to 1 and all other “EI_PORTING” flags to 0.
- Fixed classification probability: with the firmware able for classifying real-time OV7675 images, the problem occurred that the probabilities for letters A, B and C are some fixed numbers (all close to 1/3).
 - Solution: use type “unsigned short” for array “features_read”. Otherwise, using “float” type will cause image data to be recognized as very small values.
- Others: there are multiple libraries required related to the task of image classification. We added those libraries on the way (according to errors reported). An essential one is Arduino_OV767X, which is related to the camera function of the board.

Setup for Image Capture and Classification

The setup.cpp file contains initialization and main loop functions, and serves as an interface between the main function and image classification implementation. Since using the image capture initialization and processing interfaces from Edge Impulse causes error (project stuck at image capture function), we use the interfaces from OV767X library instead. The codes for initializing and main functions are shown below.

```

void ei_main_init(void)
{
    EiDeviceNanoBle33 *dev = static_cast<EiDeviceNanoBle33*>(EiDeviceInfo::get_device());
    ei_printf("Hello from Edge Impulse on Arduino Nano 33 BLE Sense\r\n"
              "Compiled on %s %s\r\n",
              __DATE__,
              __TIME__);

    // we cannot flash anymore after hitting a hard fault, so let's wait 10 seconds
    // so we can attach to the serial port
    // for (size_t ix = 0; ix < 10; ix++) {
    //     ei_printf("Waiting to start: %lu\r\n", ix);
    //     led = !led;
    //     wait_ms(1000);
    // }

    if (ei_inertial_init() == false) {
        ei_inertial_rev2_init();
    }

    if (ei_environment_init() == false) {
        ei_environment_rev2_init();
    }

    ei_interaction_init();
    // ei_camera_init();
    ei_microphone_init();

    at = ei_at_init(dev);
    ei_printf("Type AT+HELP to see a list of commands.\r\n");
    at->print_prompt();

    Serial.begin(9600);
    while (!Serial);

    Serial.println("OV767X Camera Capture");
    Serial.println();

    if (!Camera.begin(QCIF, RGB565, 1)) {
        Serial.println("Failed to initialize camera!");
        while (1);
    }

    Serial.println("Send the 'c' character to read a frame ...");
    Serial.println();
}

```

```

void ei_main()
{
    /* handle command coming from uart */

    /*
    char data = Serial.read();
    while (data != 0xFF) {
        at->handle(data);
        data = Serial.read();
    }
    */

    signal_t signal;
    signal.total_length = EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE;
    signal.get_data = &get_feature_data;

    ei_impulse_result_t result;

    if (Serial.read() == 'c') {
        Serial.println("Reading frame");
        Serial.println();
        Camera.readFrame(image);

        Serial.print("image read");
        Serial.println();

        int feature_cnt = 0;
        int col_read, row_read, col_cls, row_cls, ptr_read;
        for (int feature_cnt = 0; feature_cnt < CAMERA_WIDTH * CAMERA_HEIGHT; feature_cnt++) {
            col_cls = feature_cnt % CAMERA_WIDTH;
            row_cls = feature_cnt / CAMERA_WIDTH;
            col_read = col_cls * 176 / CAMERA_WIDTH;
            row_read = row_cls * 144 / CAMERA_HEIGHT;
            ptr_read = row_read * 176 + col_cls;
            features_read[row_cls * CAMERA_WIDTH + col_cls] = (unsigned short)(image[ptr_read]); // /
65535.0f;
            Serial.print("0x");
            Serial.print(image[ptr_read], HEX);
            Serial.print(", ");
        }

        Serial.print("features read");
        Serial.println();

        // Assuming `features_read` holds the input data (e.g., image buffer):
        //ei_printf("Free memory before: %lu\n", xPortGetFreeHeapSize());
        EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);
        //ei_printf("Free memory after: %lu\n", xPortGetFreeHeapSize());

        if (res != EI_IMPULSE_OK) {
            ei_printf("ERR: Failed to run classifier (%d)\n", res);
            return;
        }

        ei_printf("Predictions:\n");
        for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
            ei_printf("%s: %.5f\n", result.classification[ix].label, result.classification[ix].value);
        }
    }
}

```

References

[1] Edge Impulse firmware library for Arduino IDE. <https://github.com/edgeimpulse/firmware-arduino-nano-33-ble-sense>

[2] TinyMLx Image Viewer.

<https://colab.research.google.com/github/tinyMLx/colabs/blob/master/4-2-12-OV7675ImageViewer.ipynb#scrollTo=WSozbpyAFs8a>